

Semana 11: Planejamento de Testes

Por: Eduardo Neves de Souza

Squad: 01 – Unit to Unity

Data: 06/02/2026

1. Identificação do Projeto

Nome do Projeto:

Planejamento de Testes Funcionais – ServeRest

API Testada:

ServeRest – Compass UOL (ServeRest)

Versão do Plano:

1.0

Data de Criação:

02/02/2026

Data da Última Atualização:

06/02/2026

Responsável pela Elaboração:

Eduardo Neves de Souza

Responsável pela Aprovação:

Avaliadores do Programa de Bolsas

Tempo Estimado de Execução:

Em torno de 16 horas (considerando o tempo integral do estágio durante a semana)

2. Escopo

Este plano contempla a execução de **testes funcionais e testes exploratórios** na API ServeRest, direcionados aos principais fluxos da aplicação, incluindo **registro de usuários, autenticação, administração de produtos e gerenciamento de carrinhos**.

A atividade de testes tem como finalidade detectar **erros funcionais, falhas de validação e**

inconsistências nas regras de negócio, avaliando o comportamento da API por meio da análise das **requisições realizadas e das respostas retornadas**.

2.1 Funcionalidades incluídas no escopo

Usuários

- Realização das operações de criação, edição e exclusão de usuários
- Verificação do preenchimento e validação de campos obrigatórios
- Análise das regras de unicidade para evitar registros duplicados

Autenticação

- Execução de testes com dados de acesso válidos e inválidos
- Validação do processo de geração e uso de tokens de autenticação
- Avaliação das respostas da API em cenários de erro de login

Produtos

- Testes das funcionalidades de criação, consulta, atualização e remoção de produtos
- Validação dos dados enviados nos payloads das requisições
- Verificação das permissões de acesso aos endpoints
- Análise do cumprimento das regras de negócio relacionadas

Carrinhos

- Criação, finalização e cancelamento de carrinhos de compra
- Verificação das dependências entre usuários e produtos
- Avaliação do comportamento da API em situações fora do fluxo esperado

Tratamento de exceções

- Análise das respostas da API diante de entradas inesperadas
- Testes com dados inválidos ou incompletos
- Execução de requisições em sequências não previstas

3. Objetivos

3.1 Objetivo Geral

Analisar a API ServeRest por meio da execução de **testes funcionais e testes exploratórios orientados por Sabotage Tour**, com o objetivo de compreender como os endpoints se comportam diante de diferentes cenários de uso. A avaliação busca verificar a **aderência às**

regras de negócio, a **consistência no tratamento dos dados** e a **qualidade das respostas retornadas pela API** frente a entradas válidas e inválidas.

3.2 Objetivos Específicos

- Verificar se os dados recebidos pela API são **processados e validados corretamente**, respeitando obrigatoriedade, formato e tipo das informações
- Avaliar a **estabilidade e robustez dos fluxos principais**, incluindo autenticação, cadastro de usuários, produtos e carrinhos
- Analisar a **padronização e adequação dos status codes, mensagens e estruturas de resposta** retornadas pelos endpoints
- Investigar **comportamentos não documentados ou inconsistentes** por meio de testes exploratórios conduzidos com base em tours

4. Estratégia de Teste Exploratório

Estratégia adotada: Testes Exploratórios Baseados em *Sabotage Tour*

Justificativa

A estratégia de **Sabotage Tour** foi adotada com o objetivo de conduzir a exploração da API a partir de uma perspectiva **crítica e investigativa**, partindo do pressuposto de que os fluxos e validações podem conter falhas ou comportamentos não previstos.

Diferentemente de abordagens exploratórias mais amplas, a Sabotage Tour direciona os testes para **pontos sensíveis da aplicação**, como validação de entradas, dependência entre recursos, controle de acesso e cumprimento das regras de negócio. Essa estratégia incentiva a execução de requisições fora do fluxo esperado, o uso de dados inconsistentes e a combinação de cenários que buscam **expor fragilidades na implementação da API**.

A aplicação dessa abordagem permite avaliar não apenas se os endpoints funcionam em condições ideais, mas principalmente como a API reage a **entradas inválidas, sequências incorretas de chamadas, tentativas de uso indevido e cenários não documentados**. Dessa forma, torna-se possível identificar falhas de validação, inconsistências de resposta, erros de tratamento e comportamentos inesperados que dificilmente seriam detectados por testes funcionais roteirizados.

5. Mapeamento de Endpoints

Recurso	Endpoint	Método	Função	Autenticação	Regras Relevantes	Status Esperados
Login	/login	POST	Autenticar usuário	Não	Geração de token para acesso às rotas privadas	200, 401
Usuários	/usuarios	POST	Criar usuário	Não	E-mail deve ser único	201, 400
Usuários	/usuarios	GET	Listar usuários	Sim	Permite filtragem de resultados	200
Usuários	/usuarios/{_id}	GET	Buscar usuário	Sim	Usuário deve existir	200, 400
Usuários	/usuarios/{_id}	PUT	Atualizar ou criar usuário	Sim	Operação de UPSERT; valida e-mail único	200, 201, 400
Usuários	/usuarios/{_id}	DELETE	Excluir usuário	Sim	Não permite exclusão de usuário	200, 400

					com carrinho ativo	
Produtos	/produtos	POST	Criar produto	Sim (Admin)	Acesso restrito a administrador; nome do produto deve ser único	201, 400, 401, 403
Produtos	/produtos	GET	Listar produtos	Não	Permite busca por atributos	200, 400
Produtos	/produtos/{_id}	GET	Buscar produto	Não	Produto deve existir	200, 400
Produtos	/produtos/{_id}	PUT	Atualizar ou criar produto	Sim (Admin)	Operação de UPSERT; acesso restrito a administrador	200, 201, 400, 401, 403
Produtos	/produtos/{_id}	DELETE	Excluir produto	Sim (Admin)	Proibido excluir produto vinculado a carrinho	200, 400, 401, 403
Carrinhos	/carrinhos	POST	Criar carrinho	Sim	Apenas um carrinho	201, 400, 401

					por usuário; produto deve existir; reduz estoque	
Carrinhos	/carrinhos	GET	Listar carrinhos	Sim	Retorna carrinhos cadastrados	200
Carrinhos	/carrinhos/{_id}	GET	Buscar carrinho	Sim	Carrinho deve existir	200, 400
Carrinhos	/carrinhos/concluir-compra	DELETE	Concluir compra	Sim	Exclui carrinho e atualiza estoque	200, 401
Carrinhos	/carrinhos/cancelar-compra	DELETE	Cancelar compra	Sim	Exclui carrinho e reabastecer estoque	200, 401

6. Riscos e Prioridades

Área Avaliada	Ponto de Atenção	Risco Envolvido	Abordagem de Teste
Sequência de chamadas	Execução de endpoints fora da ordem esperada	Estados inválidos da aplicação	Testes exploratórios (Sabotage Tour) executando

			operações fora do fluxo padrão
Dependência entre recursos	Uso de carrinho sem produtos válidos ou usuário inexistente	Inconsistência de dados	Testes funcionais e exploratórios focados em dependências entre usuários, produtos e carrinhos
Reutilização de recursos	Reexecução de ações já concluídas (ex.: cancelar carrinho inexistente)	Comportamento imprevisível da API	Testes exploratórios validando idempotência e respostas repetidas
Manipulação de payload	Envio de campos extras, ausentes ou com tipos incorretos	Quebra de contrato da API	Testes funcionais de contrato e Sabotage Tour variando payloads
Autenticação em contexto indevido	Uso de token válido em recurso não correspondente ao usuário	Acesso indevido a dados	Testes exploratórios focados em associação entre token e recurso
Consistência de respostas	Retorno diferente para erros semelhantes	Dificuldade de consumo da API	Testes comparativos de status codes e mensagens de erro
Verbos HTTP	Uso incorreto de método	Falhas de semântica REST	Testes exploratórios

	(POST/DELETE/PUT)		utilizando verbos inadequados para cada endpoint
Persistência de dados	Dados não removidos ou atualizados corretamente	Resíduos de estado no sistema	Testes funcionais validando impacto das operações no estado da API

7. Mapeamento de Endpoints

ID	Descrição do Cenário	Endpoint	Classificação	Prioridade	Situação
CT-F01	Registrar usuário com dados válidos	POST /usuarios	Positivo	Alta	Planejado
CT-F02	Impedir cadastro com e-mail já existente	POST /usuarios	Negativo	Alta	Planejado
CT-F03	Rejeitar cadastro sem campos obrigatórios	POST /usuarios	Negativo	Alta	Planejado

CT-F04	Validar schema de request e response do cadastro	POST /usuarios	Contrato	Média	Planejado
CT-F05	Autenticar usuário com credenciais válidas	POST /login	Positivo	Alta	Planejado
CT-F06	Bloquear login com credenciais inválidas	POST /login	Negativo	Alta	Planejado
CT-F07	Retornar erro ao tentar login sem payload	POST /login	Negativo	Média	Planejado
CT-F08	Validar schema da resposta de autenticação (token)	POST /login	Contrato	Alta	Não executado
CT-F09	Criar produto com perfil administrador	POST /produtos	Positivo	Alta	Planejado
CT-F10	Impedir criação de	POST /produtos	Negativo	Alta	Planejado

	produto por usuário não autorizado				
CT-F11	Rejeitar criação de produto com dados inválidos	POST /produtos	Negativo	Média	Planejado
CT-F12	Validar schema de request e response do produto	POST /produtos	Contrato	Média	Planejado
CT-F13	Listar produtos cadastrad os com sucesso	GET /produtos	Positivo	Média	Planejado
CT-F14	Criar carrinho com produtos válidos	POST /carrinhos	Positivo	Alta	Planejado
CT-F15	Impedir criação de carrinho sem autenticaç ão	POST /carrinhos	Negativo	Alta	Planejado

CT-F16	Rejeitar carrinho com produto inexistente	POST /carrinhos	Negativo	Alta	Planejado
CT-F17	Validar schema da criação de carrinho	POST /carrinhos	Contrato	Média	Planejado
CT-F18	Concluir carrinho existente com sucesso	DELETE /carrinhos/ concluir-compra	Positivo	Alta	Planejado
CT-F19	Retornar erro ao finalizar carrinho inexistente	DELETE /carrinhos/ concluir-compra	Negativo	Média	Planejado
CT-F20	Validar estrutura da resposta ao concluir carrinho	DELETE /carrinhos/ concluir-compra	Contrato	Média	Planejado

8. Critérios de Entrada, Encerramento e Condição de Aceite

8.1 Condições para Início dos Testes

Os testes somente serão iniciados após o atendimento dos seguintes pré-requisitos:

- Disponibilidade da API em ambiente acessível para testes;
- Definição clara dos endpoints que farão parte do ciclo de testes;
- Existência de dados mínimos para execução, incluindo exemplos válidos e inválidos;

- Compreensão das regras de negócio associadas a cada recurso;
- Definição prévia do que caracteriza sucesso ou falha para cada cenário avaliado.

8.2 Condições para Encerramento dos Testes

O ciclo de testes será encerrado quando:

- Os cenários funcionais planejados tiverem sido executados;
- As sessões de testes exploratórios tiverem sido concluídas;
- Os fluxos considerados críticos estiverem avaliados;
- As anomalias encontradas estiverem registradas;
- Não existirem cenários pendentes relacionados aos fluxos prioritários.

8.3 Condição de Aceite (Definição de Pronto)

A API será considerada pronta do ponto de vista de testes quando:

- Os principais fluxos de uso estiverem validados;
- O comportamento da API estiver consistente com as regras de negócio conhecidas;
- As evidências de teste estiverem documentadas;
- As falhas relevantes identificadas estiverem registradas para acompanhamento.

9. Evidências – Exemplos de Requisições

Exemplo	Cenário	Endpoint	Método	Headers	Request Body
1	Login com credenciais válidas	https://serverest.dev/login	POST	—	{ "email": "teste@compass.uol", "password": "teste123" }
2	Login com senha inválida	https://serverest.dev/login	POST	—	{ "email": "teste@compass.uol",

					"password": "senhaErrada123" }
3	Cadastro de produto como administrador	https://serverest.dev/produtos	POST	Authorization: Bearer <token>	{ "nome": "Notebook Dell Inspiron 15", "preco": 3500, "descricao": "Notebook para desenvolvimento", "quantidade": 10 }
4	Cadastro de produto com preço negativo (Boundary)	https://serverest.dev/produtos	POST	Authorization: Bearer <token>	{ "nome": "Produto Teste", "preco": -100, "descricao": "Testando validação", "quantidade": 5 }
5	Cadastro de produto sem autenticação	https://serverest.dev/produtos	POST	—	{ "nome": "Produto Teste", "preco": 100, "descricao": "Testando validação", "quantidade": 5 }

					": "Sem token", "quantidade": 1 }
6	Cadastro de usuário com e-mail duplicado	https://serverest.dev/usuarios	POST	—	{ "nome": "Usuário Teste", "email": "teste@compass.uol", "password": "teste123", "administrador": "false" }
7	Criação de carrinho com produto inexistente	https://serverest.dev/carrinhos	POST	Authorization: Bearer <token>	{ "produtos": [{ "idProduto": "produtoFalso", "quantidade": 1 }] }